TTK4250 Group Assignment 1

Group 50: Filip Nilsen

Task 5: Simulated data

(a) Changing orientation uncertainty

Even with constant sensor noise, the orientation uncertainty can change. The ESKF's error dynamics change with orientation, acceleration and angular velocity. Its GNSS matrix changes with orientation through the lever arm. When the drone manoeuvres, the position measurements give different amounts of information about orientation and biases. The uncertainty therefore doesn't have to settle at one value. It also changes between predictions and GNSS updates. Figure 1 shows the orientation estimates, and Figure 2 shows the errors and uncertainty.

(b) Yaw error and divergence

Roll and pitch errors tilt the estimated gravity direction. This affects the predicted motion, which GNSS can help correct. Gravity doesn't tell me the heading, so yaw is harder to estimate. This explains the larger yaw error and uncertainty in Figure 2.

Without heading information, gyro noise and bias can make yaw drift and its estimated uncertainty keep growing. Standing still or flying straight at constant velocity gives little information to correct this. Losing GNSS also makes it harder. Manoeuvres with GNSS available can help limit the drift. The heading can keep drifting, even though the plotted angle wraps around at ±180°. Large errors also break the ESKF's assumption of small errors.

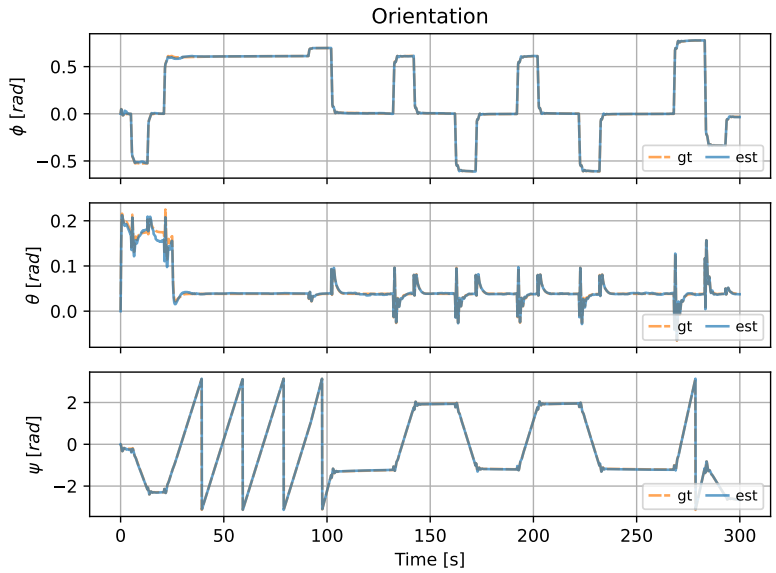


Figure 1: Baseline Euler angle estimates and ground truth with original calibration.

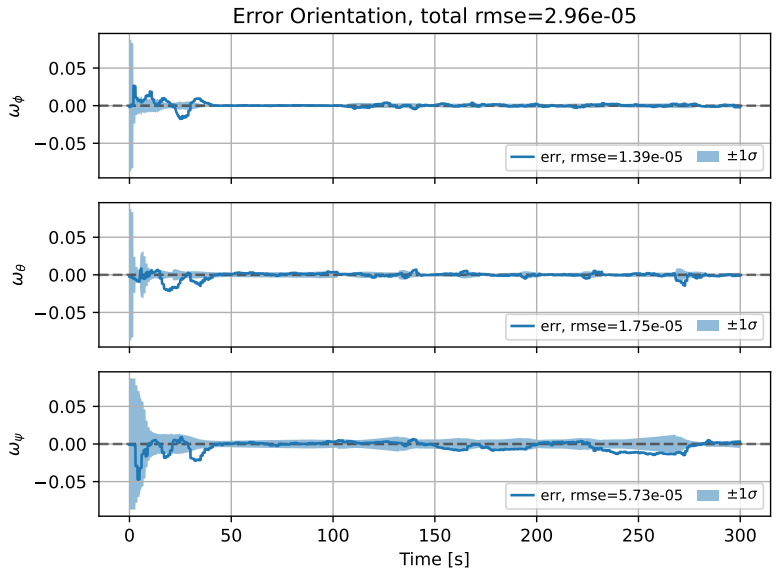


Figure 2: Baseline local orientation errors with plus/minus one standard deviation bounds.

(c) Rounded calibration matrices

I ran the first 300 seconds with the original and rounded matrices, keeping the other settings the same. Both rounded matrices became identity matrices. Position vector RMSE went from 0.46 m to 0.70 m, velocity vector RMSE from 0.19 m/s to 0.44 m/s, and yaw-direction error RMSE from 0.43 to 1.99 degrees. I checked errors every 0.1 seconds and after GNSS updates.

Rounding removes the small scale and alignment corrections. This leaves systematic IMU errors that build up during integration. GNSS limits position drift, but doesn't measure orientation directly. Estimated biases can make up for some calibration error, but not fully for errors that change with acceleration and angular velocity.

The filter also became overconfident. Its average orientation standard deviations barely changed, even though the actual errors grew. Mean 15-state NEES went from 20.7 to 1692.7. Figures 2 and 3 show the orientation errors before and after rounding. Figure 4 shows the orientation NEES after rounding.

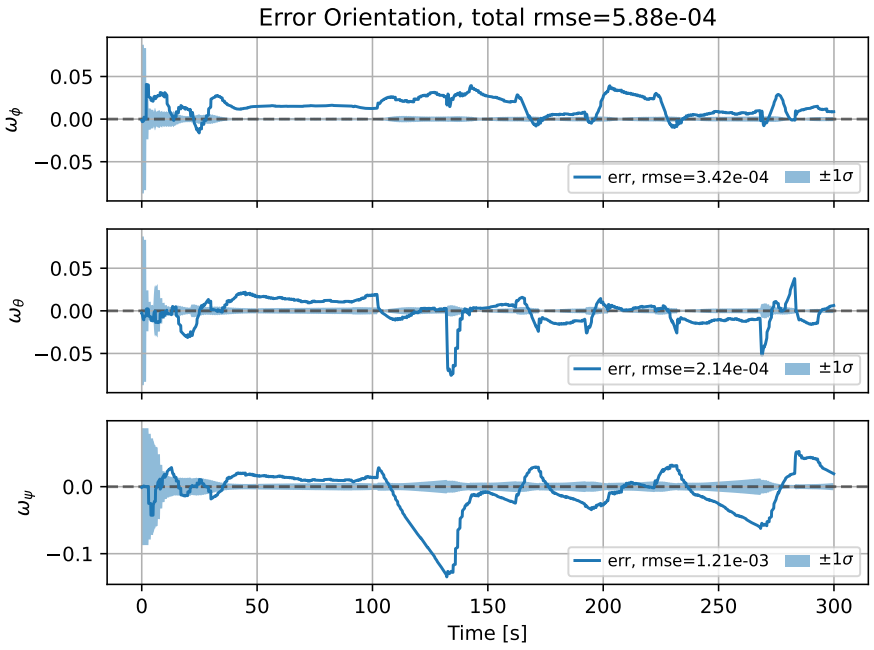


Figure 3: Local orientation errors and one standard deviation bounds with rounded calibration.

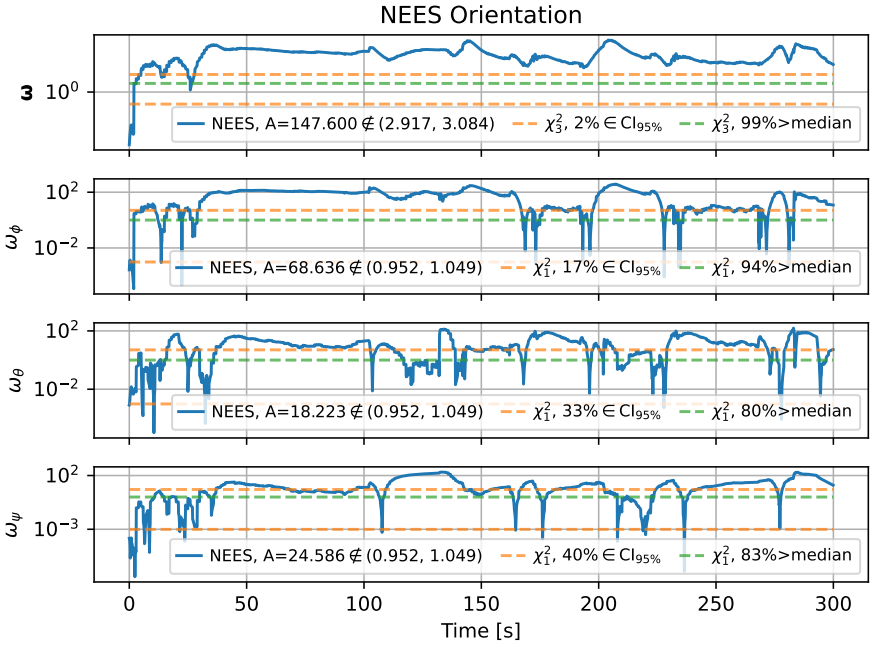


Figure 4: Orientation NEES with rounded calibration: the three-component error and individual components.

(d) Tuning experiments

I tested each change over 300 seconds with original calibration. Baseline position and velocity RMSEs were 0.46 m and 0.19 m/s, with mean NIS 2.99 and full-state NEES 20.7.

1. I reduced gnss_std_ne from 0.3 to 0.03 m and gnss_std_d from 0.5 to 0.05 m. Position and velocity RMSE rose to 0.64 m and 0.39 m/s, with mean NIS 247.2 and NEES 564.1. The filter trusted the same noisy measurements too much, making the estimates worse and overconfident.

2. I increased accm_std and gyro_std by a factor of ten, to 0.01167 and 0.000436 respectively. Position and velocity RMSE barely changed, at 0.47 m and 0.19 m/s. Mean NEES fell to 15.8, close to the expected value of 15, and NIS was 2.93. The uncertainty better matched the errors, but the estimates weren't more accurate.

3. I set gnss_min_dt_sim to 10 seconds, reducing GNSS updates from 299 to 30. Position and velocity RMSE rose to 3.16 m and 0.66 m/s as IMU errors built up between corrections. Mean NIS was still near 3 at 3.25, so a reasonable NIS doesn't necessarily mean small position errors.

Task 6: Real data

(a) GNSS between IMU timestamps

I can split the prediction at the GNSS timestamp: predict to that time, update and inject, then predict through the rest of the IMU interval. The loop already does this, using the same IMU sample for both parts and assuming constant input. If GNSS arrives after the filter has passed its timestamp, I can update a saved state at that time and rerun the later predictions and updates.

(b) Rounded calibration matrices

Over 200-600 seconds, the rounded and original runs differed by 0.27 m RMS in position. Local orientation differences were 1.37, 2.80 and 1.49 degrees RMS. These compare the two estimates, not their errors against ground truth. Mean position NIS rose from 3.65 to 4.25, and the fraction inside the 95% interval fell from 86% to 83% (Figures 5 and 6).

Without ground truth, I couldn't tell from these changes alone that the calibration was wrong. GNSS keeps position in check, and estimated biases can hide some calibration errors. NIS points to worse consistency, but doesn't tell me how accurate the orientation is or what caused the problem. For real use, I need good calibration and tests with different manoeuvres, preferably with an independent reference.

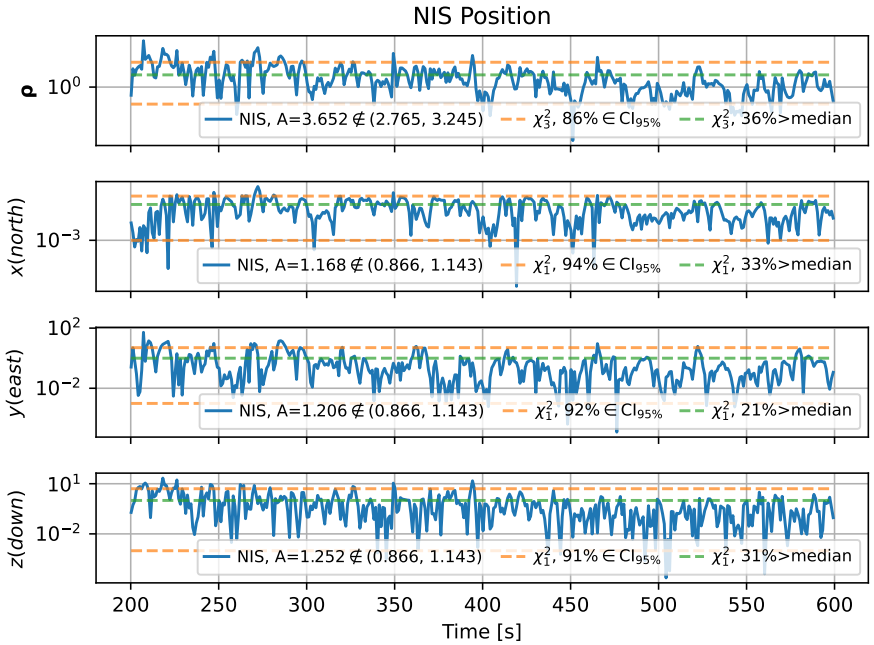


Figure 5: Real-data position NIS with original calibration, 200–600 seconds.

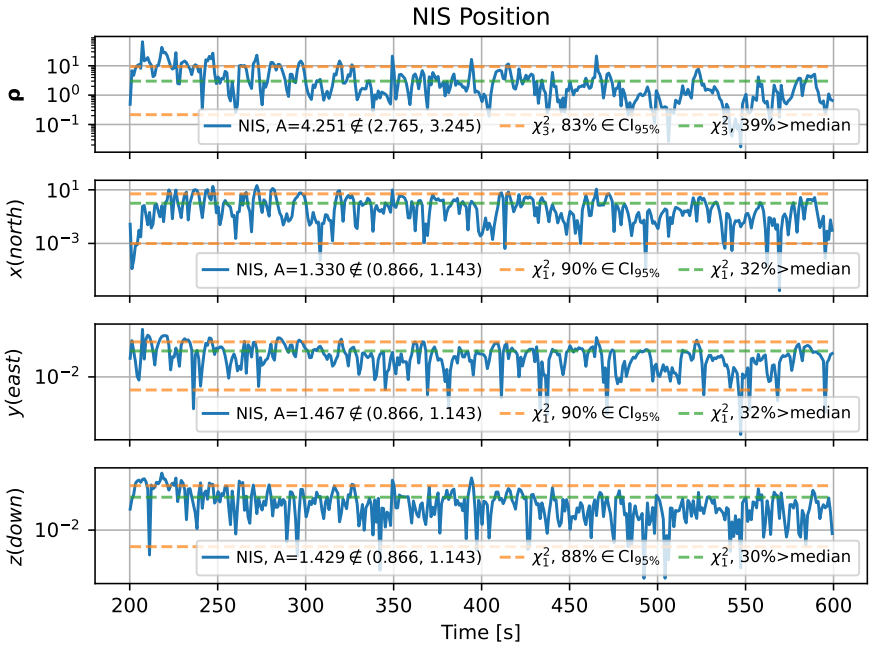


Figure 6: Real-data position NIS with rounded calibration, 200–600 seconds.

(c) Tuning without ground truth

I tested each change over 200-600 seconds with original calibration and baseline mean position NIS 3.65.

1. Reducing gnss_std_ne and gnss_std_d to 0.03 and 0.05 m raised NIS to 139.4 (Figure 7), even though RMS innovation fell from 0.96 to 0.80 m. Following GNSS more closely didn't prove better accuracy: the filter was too confident. Agreement with noisy measurements isn't enough to judge accuracy.

2. Increasing these standard deviations to 3 and 5 m reduced NIS to 0.34 (Figure 8), while RMS innovation rose to 2.80 m. The larger assumed covariance makes NIS smaller without fixing the model. I therefore can't tune just by trying to get the lowest NIS; I still need independent checks.

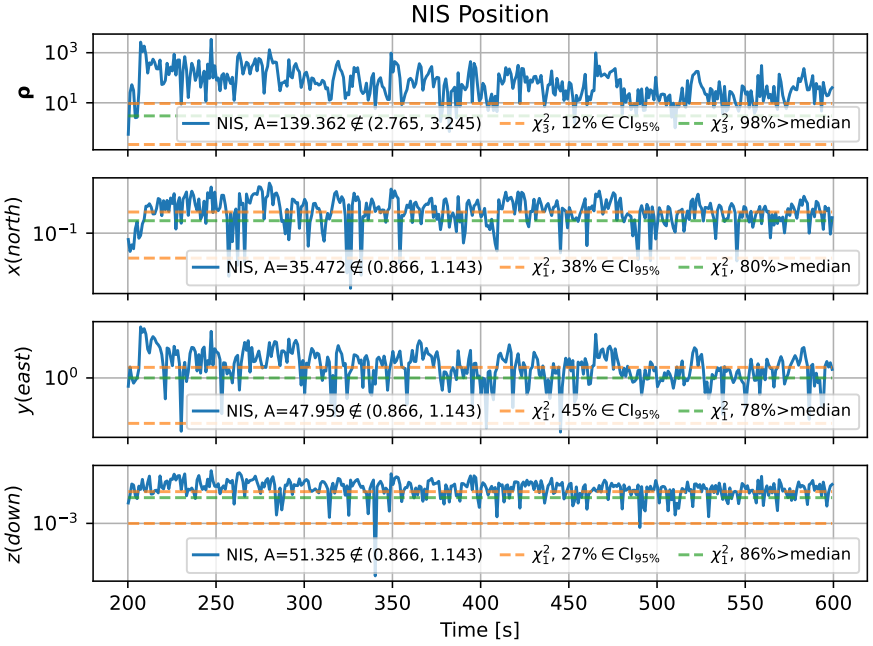


Figure 7: Real-data position NIS with GNSS standard deviations divided by ten.

3. Setting gnss_min_dt_real to 10 seconds reduced updates from 400 to 40. Mean NIS reached 71.7, mostly because of large innovations at the start (Figure 9), although 80% of updates were inside the 95% interval. Fewer corrections made it harder for the filter to settle. Reasonable innovations later on don't rule out errors between updates, which are hard to assess without ground truth.

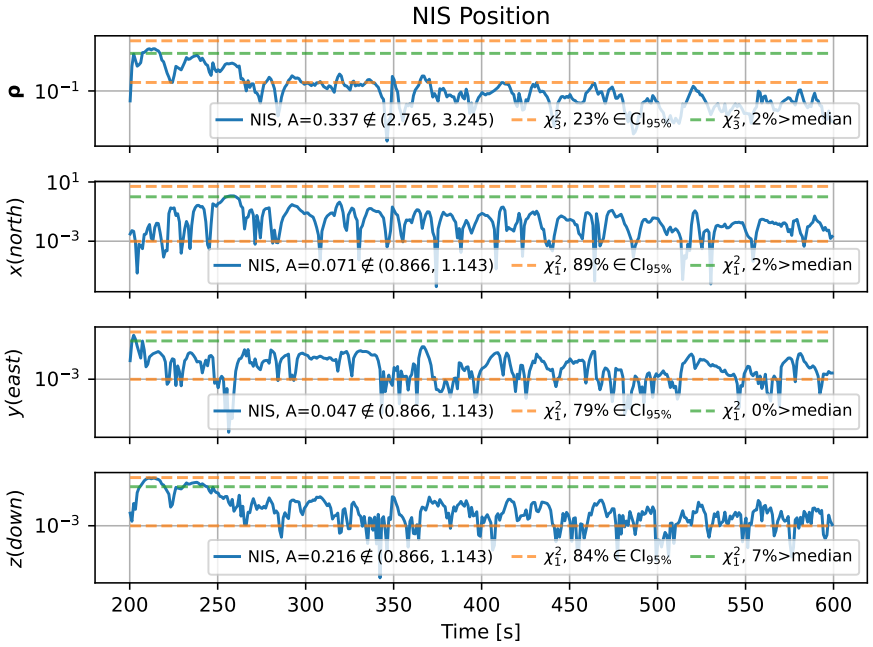


Figure 8: Real-data position NIS with GNSS standard deviations multiplied by ten.

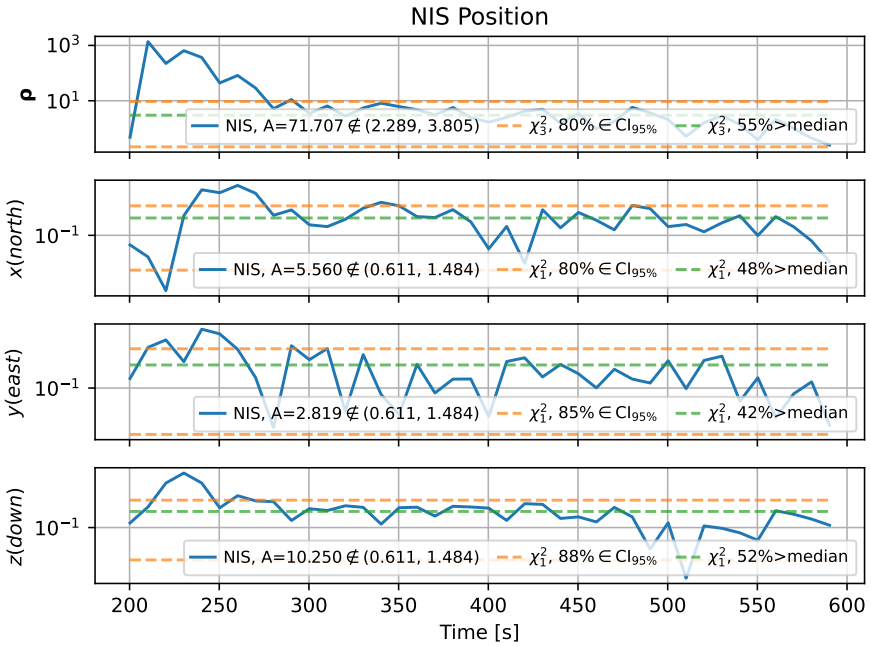


Figure 9: Real-data position NIS with GNSS updates at least 10 seconds apart.